

****Programa:**** Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo
****Alumno:**** Oscar David González Pintor – Matrícula: 2173071702
****Licenciatura:**** Ingeniería en Computación – UAM Cuajimalpa
****Responsable del proyecto:**** Dra. Alicia Montserrat Alvarado González (Núm. económico: 41051)
****Periodo:**** 1 de abril de 2026 – 2 de octubre de 2026

— — —

****Universidad Autónoma Metropolitana – Unidad Cuajimalpa****
Casa abierta al tiempo

— — —

1. La migración del entorno de simulación de ROS 2 Iron con Gazebo Clásico a **ROS 2 Jazzy Jalisco** con Gazebo Harmonic, garantizando su reproducibilidad mediante una imagen Docker pre-compilada de cinco capas.
2. El desarrollo del **nodo de Corteza Premotora** (`SimulationController`) en C++, capaz de generar trayectorias articulares reactivas y enviar su resultado a la memoria procedural del robot simulado.
3. La adaptación de las **Cortezas Somatosensorial y Parietal** al robot simulado, materializada en la configuración de siete sensores de contacto (bumpers) en Gazebo y su integración a través del bridge de comunicación.
4. La configuración de la **Corteza Motora Primaria** mediante el `JointTrajectoryController` de `ros2_control`, que traduce las órdenes del nodo cognitivo en movimiento articular ejecutado por el simulador.
5. La integración del brazo y el robot simulados en el **mismo espacio computacional**, conectando los contenedores `sim` (Gazebo) y `brain` (nodo cognitivo) a través de una red compartida y memoria IPC común.

Planeación

Calendario Oficial del Proyecto

La siguiente tabla reproduce las actividades oficiales del programa de servicio social tal como figuran en la carta de aceptación emitida el 31 de marzo de 2026. El reporte actual cubre los **Meses 1, 2 y 3**.

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5	Mes 6
Apoyo en la implementación del nodo Corteza Premotora y enviar a la memoria procedural: robot simulado	+	+				
Apoyo para levantar el brazo en ROS1						
Apoyo para migrar el robot de ROS1 a ROS2	+	+	+	+	+	+
Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado		+				
Apoyo para poner en el mismo espacio el brazo y el robot simulados				+		+
Apoyo en la documentación del proyecto	+	+	+	+	+	+
Informe trimestral			+			+
Informe final					+	

Correspondencia semana-mes para este trimestre:

Mes	Semanas	Actividades ejecutadas
1 (Abril)	1-4	Entorno Docker, xhost, estudio de Robotic-Swarms (referencia de Corteza Premotora)
2 (Mayo)	5-8	Arquitectura Xolobot, Corteza Motora Primaria (JTC), Cortezas Somatosensorial y Parietal (sensores)
3 (Junio)	9-12	Imagen Docker 5 capas, Corteza Premotora (SimulationController), integración brain+sim, VM

Justificación de la Estrategia de Containerización con Docker

Antes de iniciar el desarrollo técnico, se evaluaron las opciones de entorno disponibles. La conclusión fue que Docker era la única solución viable para cumplir los requisitos de reproducibilidad que la arquitectura cognitiva exige: los módulos deben poder ejecutarse en cualquier equipo del laboratorio sin intervención manual.

Aislamiento

Los módulos cognitivos (Corteza Premotora, Motora, Somatosensorial) se desarrollan como nodos ROS 2 independientes. Docker permite ejecutar diferentes

versiones de ROS 2 en contenedores distintos sin interferencias entre sus dependencias, lo que facilita el desarrollo paralelo de módulos.

Limpieza

El historial de instalaciones del laboratorio incluía residuos de paquetes `.deb` de software incompatible que contaminaban el entorno de ROS 2. Con Docker, cualquier estado corrupto se resuelve eliminando el contenedor y recreándolo desde la imagen, sin afectar el sistema operativo del host.

Compatibilidad

ROS 2 Jazzy Jalisco requiere Ubuntu 24.04 como base. Docker provee el sistema de archivos de Ubuntu 24.04 dentro del contenedor con independencia del sistema operativo del host, permitiendo que equipos con Ubuntu 22.04 ejecuten la arquitectura cognitiva sin reinstalación del sistema.

Portabilidad para la Arquitectura Cognitiva

La imagen Docker pre-compilada garantiza que cualquier nodo cognitivo futuro (Corteza Prefrontal, Hipocampo, etc.) pueda incorporarse al sistema ejecutando un único comando de construcción, sin que diferencias de plataforma afecten la ejecución.

Inicio

Al comenzar el trimestre se verificó que el entorno preexistente de ROS 2 Iron era funcional en el equipo original pero no portable. Se estableció como primer hito la obtención de un entorno Docker completamente operativo antes de proceder con la implementación de los módulos cognitivos.

Desarrollo

Semana 1

Proceso: Preparación del entorno gráfico y diagnóstico del sistema

La arquitectura cognitiva requiere un simulador 3D para ejecutar el robot en su entorno. Gazebo Harmonic, el simulador objetivo, utiliza el protocolo X11 para renderizar su interfaz gráfica. El primer obstáculo técnico fue configurar el forwarding de esta interfaz hacia el contenedor Docker.

Paso A: Habilitar el acceso gráfico (crítico para Gazebo)

Sin esta configuración, Gazebo termina con el siguiente error al intentar inicializar su renderer:

```

```
qt.qpa.xcb: could not connect to display :0
qt.qpa.plugin: Could not load the Qt platform plugin "xcb"
[ERROR] [gz-5]: process has died [pid 8423, exit code 1]
```
```

La solución es autorizar al demonio Docker para usar el socket X11 del host antes de lanzar cualquier contenedor:

```
```bash
xhost +local:docker
```

...

Este comando modifica la lista de control de acceso del servidor X11, permitiendo conexiones desde procesos locales no autenticados. Debe ejecutarse una vez por sesión de trabajo.

#### #### Paso B: Verificación de Docker Engine

```
```bash
docker --version
# Docker version 29.6.1, build 8900f1d

docker compose version
# Docker Compose version v5.3.0
```
```

Se confirmó que el usuario no pertenecía al grupo `docker`, lo que requería `sudo` en todos los comandos. Se corrigió con:

```
```bash
sudo usermod -aG docker $USER
newgrp docker
```
```

---

### ## Semana 2

#### ### Investigación: Estructura de las imágenes ROS 2 en Docker

Se estudió la organización de las imágenes oficiales de OSRF para comprender cómo construir el entorno cognitivo de forma estratificada. La jerarquía base es:

```
...
osrf/ros:jazzy-desktop
├── Ubuntu 24.04 (Noble Numbat)
│ └── ROS 2 Jazzy Jalisco (tools + desktop)
│ ├── Gazebo Harmonic + ros2-control + bridge
│ └── Workspace compilado (nodos cognitivos Xolobot)
...
```

Se estableció el principio de **\*\*workspace pre-compilado\*\***: la imagen compila todos los nodos cognitivos durante el `docker build`, haciendo que el arranque del sistema sea instantáneo (`docker compose up`). Esto es crítico para las sesiones de laboratorio, donde el tiempo de setup debe ser mínimo.

---

### ## Semana 3

#### ### Configuración del repositorio y control de versiones

Se organizó el repositorio Git con una estrategia de ramas por área funcional:

```
Rama	Módulo cognitivo correspondiente
`main`	Código estable de producción
`fix/cinematica-estable`	Correcciones de Corteza Motora Primaria
`feat/agarre-autonomo-waypoint`	Corteza Premotora – algoritmo de agarre reactivo
`infra/docker-gazebo-harmonic`	Infraestructura de integración (mismo espacio brain+sim)
```

Se implementó el script `setup\_alias.sh` para automatizar la configuración del entorno en nuevas máquinas. El script es idempotente: verifica la existencia del bloque antes de escribir, evitando duplicados en `~/.bashrc`.

---

## ## Semana 4

### ### Antecedente: Proyecto Robotic-Swarms – Referencia de la Corteza Premotora

Como primera actividad formal dentro del Mes 1 del calendario oficial ("Apoyo en la implementación del nodo Corteza Premotora"), se estudió y ejecutó el proyecto **Robotic-Swarms**, que implementa una arquitectura cognitiva completa sobre ROS 2 para robótica de enjambre. Su paquete `cognitive\_architecture` contiene nodos de planificación de acción que replican el comportamiento de la Corteza Premotora en un contexto de múltiples robots, y constituyó el **referente directo** del módulo que se implementaría para el Xolobot en la Semana 10.

El proceso documentado para ejecutar Robotic-Swarms bajo ROS 2 Foxy es el siguiente:

```
```bash
docker pull osrf/ros:foxy-desktop

docker run -it --rm \
  --net=host \
  -e DISPLAY=$DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -v $(pwd):/root/ros2_ws/src/Robotic-Swarms-main \
  osrf/ros:foxy-desktop
```
```

### #### Dentro del contenedor – Compilación y ejecución del módulo cognitivo

```
```bash
source /opt/ros/foxy/setup.bash
apt-get update
cd /root/ros2_ws/
rosdep install --from-paths src --ignore-src -r -y
colcon build
source install/setup.bash
ros2 launch cognitive_architecture gazebo.launch.py
```
```

**Observación sobre la Corteza Premotora en este contexto:** El paquete `cognitive\_architecture` implementa nodos de planificación que procesan el estado del entorno y generan comandos motores. Esta estructura – un nodo que recibe señales sensoriales y produce trayectorias articulares – es exactamente la que se replicaría en `SimulationController.cpp` para el Xolobot.

---

## ## Semana 5

### ### Análisis comparativo: ROS Noetic vs ROS 2 Foxy/Jazzy en el contexto de la arquitectura cognitiva

|                         |                    |                                 |
|-------------------------|--------------------|---------------------------------|
| Aspecto                 | ROS 1 Noetic       | ROS 2 Foxy/Jazzy                |
| ---  ---  ---           |                    |                                 |
| Compilador              | `catkin_make`      | `colcon build`                  |
| Entorno compilado       | `devel/setup.bash` | `install/setup.bash`            |
| Comunicación internodos | XMLRPC + rosmaster | DDS (Data Distribution Service) |

```
| Ciclo de vida de nodos | Ninguno (solo start/stop) | Lifecycle nodes
(unconfigured → active) |
| Soporte a largo plazo | EOL Mayo 2025 | Activo – Jazzy LTS hasta 2029 |
| Idoneidad para arq. cognitiva | Limitada | Óptima – lifecycle permite
inicialización ordenada de cortezas |
```

Esta comparativa justificó técnicamente la decisión de desarrollar la arquitectura cognitiva del Xolobot exclusivamente sobre ROS 2 Jazzy.

---

## Semana 6

### Estudio del proyecto base Xolobot en ROS 2 Iron

Se procedió a estudiar la versión original del Xolobot bajo ROS 2 Iron y Gazebo Clásico. Errores catalogados durante la configuración inicial:

```
Error	Causa	Solución
`Cannot locate rosdep definition`	Terminal sin `source` de ROS 2	Activar
entorno antes de `rosdep`		
`Duplicate package names`	Paquetes en `src/` y `src_lata/` simultáneamente	
`touch src_lata/COLCON_IGNORE`		
`Package 'xolobot_arm' not found`	`install/setup.bash` no activado	`source
install/setup.bash` post-build		
`robot_description MUST not be empty`	Launch file incorrecto	Usar
`xolobot_arm_control.launch.py` |
```

Este relevamiento estableció la línea base a partir de la cual se ejecutaría la migración a Jazzy/Harmonic.

---

## Semana 7

### Corteza Motora Primaria – Configuración del JointTrajectoryController

En la arquitectura cognitiva inspirada en neuronas espejo, la **Corteza Motora Primaria** traduce los planes de acción generados por la Corteza Premotora en señales motoras dirigidas a los efectores. En el sistema Xolobot, este rol lo cumple el **JointTrajectoryController** (JTC) de `ros2_control`.

**Estructura articular del Xolobot (21 DOF):**

```
Índice	Joint	Segmento
0	`jnt_pecho_hombro`	Rotación de base
1	`jnt_hombro_hombro`	Abducción lateral
2	`jnt_hombro_biceps`	Flexión del hombro
3	`jnt_biceps_codo`	Flexión del codo
4	`jnt_codo_antebrazo`	Pronación/supinación
5	`jnt_antebrazo_palma`	Flexión de muñeca
6-8	Pulgar (3 falanges)	Mano
9-11	Índice (3 falanges)	Mano
12-14	Cordial (3 falanges)	Mano
15-17	Anular (3 falanges)	Mano
18-20	Meñique (3 falanges)	Mano
```

**Problema crítico resuelto – paquetes de ros2-control faltantes:**

```
``bash
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers
ros2 daemon stop && ros2 daemon start
```

```
ros2 control list_controllers
joint_trajectory_controller active
```
```

Semana 8

Cortezas Somatosensorial y Parietal – Sensores de contacto y bridge de comunicación

En la arquitectura cognitiva, la **Corteza Somatosensorial** procesa la información táctil del cuerpo, mientras que la **Corteza Parietal** integra esa información para construir un modelo espacial de la interacción con el entorno. Para el Xolobot simulado, estos módulos se implementan como **siete sensores de contacto (bumpers)** distribuidos en la mano y el antebrazo.

Migración de plugins SDF – diferencia crítica entre versiones de Gazebo:

```
```xml
<!-- Gazebo Clásico (Iron) -->
<plugin name="gazebo_ros_control" filename="libgazebo_ros_control.so"/>

<!-- Gazebo Harmonic (Jazzy) – plugins nativos gz-sim -->
<plugin filename="gz-sim-physics-system"
 name="gz::sim::systems::Physics"/>
<plugin filename="gz-sim-sensors-system"
 name="gz::sim::systems::Sensors">
 <render_engine>ogre2</render_engine>
</plugin>
```
```

Semana 9

Integración en el mismo espacio – Imagen Docker de 5 capas

Poner el brazo y el robot simulados "en el mismo espacio" implica que los nodos cognitivos (`brain``) y el simulador físico (`sim``) puedan comunicarse como si estuvieran en la misma máquina, con latencias mínimas y descubrimiento automático de tópicos.

docker-compose.yml – definición del espacio compartido:

```
```yaml
services:
 sim: # El cuerpo: Gazebo + JTC (Corteza Motora Primaria)
 image: xolobot:jazzy
 network_mode: host
 command: ros2 launch xolobot_arm xolobot_arm_control.launch.py

 brain: # La mente: SimulationController (Corteza Premotora)
 image: xolobot:jazzy
 network_mode: host
 depends_on:
 - sim
 command: >
 ros2 run xolobot_arm_server xolobot_arm_server
 --ros-args -p use_sim_time:=true
```
```

Semana 10

Corteza Premotora – Implementación del nodo SimulationController

La **Corteza Premotora** es la región cortical responsable de la planificación y selección de acciones motoras complejas antes de que sean ejecutadas. Para el Xolobot, la Corteza Premotora se implementa como el nodo C++ ``SimulationController``.

Arquitectura de la máquina de estados reactiva:

```
...
[INICIO] → warm-up (4 ticks × 2.5s = 10s para estabilizar /clock)
      ↓
[PLANIFICACIÓN] → Corteza Premotora genera trayectoria de 2 puntos
      ↓
[EJECUCIÓN] → Corteza Motora Primaria interpola hacia la lata
      ↓
[CONTACTO] → Corteza Somatosensorial detecta colisión palma/dedo
      ↓
[AGARRE] → Cierre de dedos + activación de articulación magnética
      ↓
[MEMORIA PROCEDURAL] → Corteza Premotora envía trayectoria de elevación
...
```

Valores de la pose de agarre (calibrados con Teach Pendant en RQT):

| Joint | Valor (rad) | Descripción biomecánica |
|-----------------------|-------------|---|
| `jnt_pecho_hombro` | 0.159 | Rotación del hombro hacia el objeto |
| `jnt_hombro_hombro` | 0.0 | Alineación lateral |
| `jnt_hombro_biceps` | -1.2874 | Flexión del hombro (valor calibrado) |
| `jnt_biceps_codo` | 1.5010 | Extensión del codo |
| `jnt_codo_antebrazo` | -1.068 | Pronación del antebrazo |
| `jnt_antebrazo_palma` | -1.1309 | Flexión de muñeca (-64.8°, perpendicular al objeto) |

Semana 11

Pruebas de integración en Máquina Virtual

Al replicar el entorno en una máquina virtual para validar la portabilidad, se presentaron cuatro problemas de infraestructura:

| Problema | Síntoma original | Solución aplicada |
|-------------------------------------|---|-------------------|
| Colapso gráfico de Gazebo | `could not connect to display :0`
`~/Xauthority` portable en compose | |
| Bloqueo DDS en red de VM | `/clock` no visible desde `brain`
`ROS_LOCALHOST_ONLY=1` | |
| Aislamiento IPC entre contenedores | JTC ignoraba trayectorias silenciosamente
`ipc: host` en ambos servicios | |
| Colisión cinemática en aproximación | Antebrazo chocaba con pedestal
Waypoint alto intermedio | |

Semana 12

Memoria Procedural – Adaptación del plan motor por retroalimentación del entorno

...

[illegible]

...

— — —

— — —

Parte III – Segundo Reporte Trimestral (Meses 4-6)

Trimestre 26-P – Segundo Reporte de Servicio Social

****Programa:**** Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo

****Alumno:**** Oscar David González Pintor – Matrícula: 2173071702

****Licenciatura:**** Ingeniería en Computación – UAM Cuajimalpa

****Responsable del proyecto:**** Dra. Alicia Montserrat Alvarado González (Núm. económico: 41051)

****Periodo reportado:**** 1 de julio de 2026 – 30 de septiembre de 2026 (Meses 4, 5 y 6)

Introducción

El presente reporte trimestral documenta las actividades realizadas durante los meses cuatro, cinco y seis del servicio social, inscrito bajo el programa ****Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo****. Durante este periodo, el trabajo se desplazó de la implementación inicial de los módulos cognitivos –completada en el trimestre anterior– hacia su ****validación, consolidación y transferencia****.

El sustrato técnico sobre el que opera la arquitectura fue sometido a pruebas de reproducibilidad en hardware independiente, se reorganizó el repositorio oficial del proyecto para facilitar su continuidad por nuevos integrantes del laboratorio, y se ejecutó el traspaso formal del entorno a un alumno de incorporación reciente. Paralelamente, se resolvieron los problemas de infraestructura que quedaban pendientes del trimestre anterior y se gestionó la estabilidad del ambiente de desarrollo ante restricciones de almacenamiento críticas.

Antecedentes

El primer reporte trimestral (Meses 1-3, abril-junio 2026) estableció la infraestructura computacional sobre la que opera la arquitectura cognitiva. Los logros de ese periodo fueron:

- Migración del entorno de simulación de ROS 2 Iron/Gazebo Clásico a ****ROS 2 Jazzy Jalisco con Gazebo Harmonic****, encapsulado en una imagen Docker de 5 capas reproducible.
- Implementación de la ****Corteza Premotora**** (``SimulationController.cpp``) como nodo C++ capaz de planificar trayectorias reactivas y detectar contacto a través de 7 bumpers.
- Configuración de la ****Corteza Motora Primaria**** (JTC de ``ros2_control``) y de las ****Cortezas Somatosensorial y Parietal**** (bridge YAML con Gazebo Harmonic).
- Resolución de los cuatro problemas de infraestructura en VM: forwarding gráfico X11, sincronización DDS, aislamiento IPC y colisión cinemática con waypoint.
- Validación de la secuencia completa de agarre autónomo: aproximación → contacto → cierre de dedos → elevación.

El presente trimestre parte de ese estado funcional y se enfoca en su robustez, portabilidad y continuidad.

Objetivo

Consolidar y transferir la Arquitectura Cognitiva Inspirada en Neuronas Espejo para el robot Xolobot, mediante:

1. La validación de la reproducibilidad del entorno en hardware independiente, mediante pruebas de estrés en una instalación limpia de Ubuntu 24.04.

2. La reorganización del repositorio oficial del proyecto en un repositorio dedicado (`xolobot-cognitive-core`), con documentación maestra que garantice la incorporación autónoma de nuevos integrantes.
3. La gestión de la infraestructura de desarrollo: resolución de conflictos de dependencias, optimización del almacenamiento en disco y expansión de la capacidad de almacenamiento disponible.
4. El traspaso formal del entorno al alumno Diego Vázquez, en coordinación con la Dra. Montserrat.

Planeación

Calendario Oficial del Proyecto

La siguiente tabla reproduce las actividades oficiales del programa de servicio social tal como figuran en la carta de aceptación emitida el 31 de marzo de 2026. El reporte actual cubre los ****Meses 4, 5 y 6****.

| Actividad | Mes 1 | Mes 2 | Mes 3 | Mes 4 | Mes 5 | Mes 6 |
|--|-------|-------|-------|-------|-------|-------|
| Apoyo en la implementación del nodo Corteza Premotora y enviar a la memoria procedural: robot simulado | + | + | + | + | + | + |
| Apoyo para levantar el brazo en ROS1 | + | + | + | + | + | + |
| Apoyo para migrar el robot de ROS1 a ROS2 | + | + | + | + | + | + |
| Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado | + | + | + | + | + | + |
| Apoyo para poner en el mismo espacio el brazo y el robot simulados | + | + | + | + | + | + |
| Apoyo en la documentación del proyecto | + | + | + | + | + | + |
| Informe trimestral | + | + | + | + | + | + |
| Informe final | + | + | + | + | + | + |

****Correspondencia semana-mes para este trimestre:****

| Mes | Semanas | Actividades ejecutadas |
|----------------|---------|---|
| 4 (Julio) | 13-16 | Validación Docker en VM, reorganización del repositorio oficial, documentación maestra |
| 5 (Agosto) | 17-20 | Prueba Acid Test en `ss-neuro`, resolución de conflictos de dependencias, gestión de almacenamiento |
| 6 (Septiembre) | 21-24 | Expansión de almacenamiento NTFS, documentación de handover, onboarding de Diego Vázquez, informe final |

Inicio

El trimestre comenzó con el entorno cognitivo funcionando sobre el equipo principal de desarrollo. El reto de esta fase no era ya implementar, sino ****garantizar que lo implementado pudiera sobrevivir fuera de ese equipo****: en una máquina diferente, con un usuario nuevo, sin la memoria contextual del desarrollador original. Esta distinción —de implementación a reproducibilidad— define la naturaleza del trabajo documentado a continuación.

Desarrollo

Semanas 13-15

Consolidación de la integración brain+sim – Docker en entorno de VM

Los cuatro problemas resueltos en las Semanas 11-12 del trimestre anterior fueron verificados y confirmados como estables:

| Problema | Síntoma original | Solución aplicada | Estado |
|-------------------------------------|---|------------------------------------|-----------|
| Colapso gráfico de Gazebo | `could not connect to display :0` | ~/.Xauthority` portable en compose | ✓ Estable |
| Bloqueo DDS en red de VM | `/clock` no visible desde `brain` | ROS_LOCALHOST_ONLY=1 | ✓ Estable |
| Aislamiento IPC entre contenedores | JTC ignoraba trayectorias silenciosamente | `ipc: host` en ambos servicios | ✓ Estable |
| Colisión cinemática en aproximación | Antebrazo chocaba con pedestal | Waypoint alto intermedio | ✓ Estable |

Se ejecutó el flujo completo desde imagen limpia para confirmar que no existían dependencias ocultas del entorno del desarrollador:

```
```bash
docker compose build --no-cache
xhost +local:docker
docker compose up sim
En segunda terminal:
docker compose run brain
```
```

****Resultado:**** La secuencia completa de agarre autónomo se ejecutó correctamente desde la imagen recién construida.

Semanas 16-18

Repositorio oficial – `xolobot-cognitive-core`

El repositorio original del proyecto (`mi-brazo-robot-ros2-iron`) había sido creado durante la fase de migración y conservaba en su nombre, estructura y ramas el rastro de ese proceso. Se creó el repositorio `xolobot-cognitive-core` como repositorio limpio y dedicado que representa el estado actual del sistema.

```
```bash
Repositorio anterior – preservado solo localmente
cd ~/migration_ws/src/mi-brazo-robot-ros2-iron
touch COLCON_IGNORE
git remote remove origin

Nuevo repositorio oficial
cd ~/migration_ws/src/xolobot-cognitive-core
git remote add origin https://github.com/oscargop98/xolobot-cognitive-core.git
git push -u origin main
```
```

Resolución del conflicto de versiones de `libfastcdr`

`apt upgrade` actualizó `libfastcdr` de la versión `2.2.7` a `2.2.8`. El directorio `build/` contenía entradas de `CMakeCache.txt` con la ruta codificada a `libfastcdr.so.2.2.7`, que ya no existía en el sistema.

****Solución:**** Reconstrucción limpia eliminando los artefactos de build previos:

```
```bash
cd ~/migration_ws
rm -rf build/ install/ log/
colcon build
```
```

```
source install/setup.bash
```
```

**\*\*Aprendizaje documentado:\*\*** Después de ejecutar `apt upgrade` en cualquier equipo donde corre el proyecto, es necesario realizar un rebuild limpio. Esto aplica también a entornos del laboratorio que reciban actualizaciones automáticas.

#### #### README maestro y documentación de arquitectura

Se redactó el `README.md` maestro del repositorio con la estructura definitiva para nuevos integrantes. Como complemento, se generó un documento técnico de apoyo para el traspaso del proyecto (`DEV\_CONTEXT.md`), que consolida la arquitectura del sistema, el estado actual del código y las decisiones de diseño relevantes para quien continúe el desarrollo en fases posteriores.

---

#### ## Semanas 19-21

##### ### Acid Test – Validación en hardware independiente (`ss-neuro`)

La pregunta que define este bloque es directa: **\*\*¿puede cualquier integrante del laboratorio, partiendo de una máquina limpia, llegar a tener el sistema corriendo siguiendo solo el README?\*\***

##### #### Configuración del equipo de prueba

Se habilitó una laptop denominada `ss-neuro` como equipo de prueba independiente. Las condiciones de la prueba fueron deliberadamente restrictivas para reflejar el escenario de un nuevo integrante:

- Instalación limpia de **\*\*Ubuntu 24.04 LTS\*\*** (sin configuraciones previas del proyecto).
- Hardware con recursos moderados (CPU de generación anterior, RAM limitada).
- Sin historial de ROS 2 en el sistema.

[Insertar imagen: Laptop ss-neuro durante la instalación de Ubuntu 24.04 – equipo de prueba sin configuración previa]

##### #### Prueba de estrés – `colcon build` en hardware limitado

La compilación tomó entre **\*\*2 y 3 horas\*\***, manteniendo todos los núcleos de la CPU a su máxima capacidad de forma sostenida. Este comportamiento es esperado y no indica un error: el ecosistema ROS 2 Jazzy con Gazebo Harmonic requiere compilar cientos de unidades de traducción con dependencias encadenadas.

[Insertar imagen: Monitor del sistema mostrando CPU al 100% durante colcon build en ss-neuro – prueba de estrés de 2-3 horas]

##### #### Validación final – Gazebo Harmonic en ejecución

Una vez completada la compilación, se ejecutó el sistema siguiendo exactamente los pasos del README:

```
```bash
source ~/migration_ws/install/setup.bash
bash ~/migration_ws/src/xolobot-cognitive-core/setup_aliases.sh
source ~/.bashrc
```

Terminal 1 – Simulador

xolo_sim

Terminal 2 – Nodo cognitivo

xolo_brain
```

**\*\*Resultado:\*\*** Gazebo Harmonic renderizó el brazo robótico Xolobot, el pedestal y la lata sin errores. El nodo `SimulationController` se sincronizó con el tiempo de simulación y completó la secuencia de agarre autónomo.

[Insertar imagen: Clonación del repositorio xolobot-cognitive-core en ss-neuro – terminal mostrando git clone exitoso]

[Insertar imagen: Gazebo Harmonic ejecutándose en ss-neuro – brazo Xolobot y lata renderizados correctamente]

**\*\*Conclusión del Acid Test:\*\*** El entorno es **\*\*reproducible\*\***. Un usuario nuevo, partiendo de Ubuntu 24.04 limpio y siguiendo la documentación del repositorio, llega a tener el sistema corriendo sin necesidad de asistencia del desarrollador original.

---

## Semanas 22–24

### Gestión de infraestructura y traspaso formal

#### Por qué el proyecto demanda tanto almacenamiento

Una de las condiciones que caracterizaron el final de este trimestre fue la gestión de un problema de espacio en disco crítico: la partición principal de Ubuntu (`/`) llegó al **\*\*97% de ocupación\*\***, con solo 5 GB libres de los 198 GB asignados.

| Componente                              | Espacio aproximado | Descripción                                                                |
|-----------------------------------------|--------------------|----------------------------------------------------------------------------|
| ROS 2 Jazzy Desktop (`/opt/ros/jazzy/`) | 3–4 GB             | Framework DDS, `rviz2`, `rqt`, herramientas CLI y dependencias             |
| Gazebo Harmonic (`gz-harmonic`)         | 4–6 GB             | Motor de física (Bullet/DART), renderizado Ogre2, modelos 3D base          |
| Caché de compilación (`build/`)         | 1–2 GB             | Archivos objeto `.o`, `CMakeCache.txt`, artefactos de Ninja – regenerables |
| Imágenes Docker (`/var/lib/docker/`)    | 8–12 GB por imagen | Contiene todo lo anterior pre-compilado; varias versiones acumulan más     |
| Logs de ejecución (`~/ros/log/`)        | 0.5–2 GB           | Crece con cada sesión de trabajo; se puede limpiar con `rm -rf ~/ros/log/` |

La suma de estas cinco capas explica cómo un proyecto de robótica simulada puede consumir entre **\*\*20 y 35 GB\*\*** de almacenamiento activo, y por qué una partición de 200 GB que parecía generosa al momento de la instalación resultó insuficiente al alcanzar la fase de pruebas intensivas.

#### Gestión del espacio en disco – recuperación de 13 GB

Con la partición al 97%, se realizó un análisis de los directorios con mayor consumo y se liberó espacio de forma selectiva, preservando todo lo relacionado con el proyecto ROS 2:

```bash

df -h /

| # Filesystem | Size | Used | Avail | Use% |
|--------------|------|------|-------|------|
| # /dev/sdb3 | 198G | 191G | 5.0G | 97% |

| | |
|----------------------|--|
| rm -rf ~/ros2_jazzy/ | # workspace compilado desde fuente – ya no necesario |
| rm -rf ~/.ros/log/ | # logs de ejecuciones pasadas |
| sudo apt clean | # caché de apt |

```
df -h /
# Filesystem      Size  Used Avail Use%
# /dev/sdb3        198G  150G   37G   81%
````
```

**\*\*Resultado:\*\*** Se recuperaron aproximadamente 13 GB, llevando la partición del 97% al 81% de ocupación con 37 GB libres.

#### #### Expansión de almacenamiento – Partición NTFS como volumen auxiliar

Se eligió la opción sin riesgo: montar permanentemente la partición NTFS con `ntfs-3g`, haciendo disponibles los 529 GB como volumen auxiliar en `/mnt/datos`:

```
```bash
sudo blkid /dev/sdb2
# /dev/sdb2: TYPE="ntfs" UUID="124A39244A39064F"

sudo mkdir -p /mnt/datos
echo "UUID=124A39244A39064F /mnt/datos ntfs-3g \
    defaults,uid=1000,gid=1000,umask=022,nofail 0 0" \
| sudo tee -a /etc/fstab

sudo mount -a
df -h /mnt/datos
# /dev/sdb2      733G  204G  529G   28%   /mnt/datos
```
```

**\*\*Resultado:\*\*** 529 GB adicionales disponibles de forma permanente en `/mnt/datos`, sin modificar la tabla de particiones.

#### #### Traspaso formal – Onboarding de Diego Vázquez

**\*\*Fecha de inicio:\*\*** 28 de septiembre de 2026

En coordinación con la Dra. Montserrat, se inició el traspaso del entorno actualizado al alumno Diego Vázquez, quien continuará con la siguiente fase operativa del proyecto.

**\*\*Diagnóstico de prerequisites:\*\*** Se detectó que Diego contaba con Ubuntu 22.04, versión incompatible con ROS 2 Jazzy Jalisco (que requiere Ubuntu 24.04 como base). Se acordó la creación de una partición dedicada para instalar Ubuntu 24.04.

**\*\*Materiales entregados:\*\***

- URL del repositorio oficial: ``https://github.com/oscaregop98/xolobot-cognitive-core``
- Enlace a la documentación oficial de instalación de ROS 2 Jazzy
- Instrucción de lectura previa del ``README.md`` maestro antes de ejecutar cualquier comando

---

#### # Observaciones y Conclusiones

##### ## Observaciones técnicas

1. **\*\*La reproducibilidad es una propiedad que debe construirse, no asumirse.\*\***  
El Acid Test demostró que un entorno que funciona en el equipo del desarrollador no es automáticamente reproducible en otro hardware.
2. **\*\*El ecosistema ROS 2 + Gazebo Harmonic tiene un coste de almacenamiento inherentemente alto.\*\*** La combinación de librerías de simulación física, modelos 3D y herramientas de visualización produce artefactos que consumen decenas de

gigabytes. Esto debe anticiparse al dimensionar cualquier máquina de desarrollo nueva.

3. **\*\*La reorganización del repositorio es una inversión en continuidad del proyecto.\*\*** El nuevo repositorio `xolobot-cognitive-core` es el punto de partida para quien se incorpore al proyecto en fases futuras.

4. **\*\*La invalidación de `CMakeCache` por actualizaciones de sistema es un patrón recurrente.\*\*** El procedimiento `rm -rf build/ install/ log/ && colcon build` debe tratarse como la respuesta estándar ante cualquier fallo de linking después de un `apt upgrade`.

5. **\*\*Montar el volumen NTFS como solución de almacenamiento fue la elección correcta.\*\*** Las alternativas de redimensionamiento de particiones conllevaban riesgo de pérdida de datos en una máquina que ya contiene trabajo en curso.

## ## Conclusiones

Al término de los Meses 4, 5 y 6 del calendario oficial, el proyecto completó su transición de la fase de implementación a la fase de **\*\*validación y transferencia\*\***.

| Actividad                             | Resultado                                                                                       | Semanas |  |
|---------------------------------------|-------------------------------------------------------------------------------------------------|---------|--|
| --- --- ---                           |                                                                                                 |         |  |
| Validación Docker en VM (4 problemas) | Todos los problemas confirmados como estables; ciclo completo de arranque verificado            | 13-15   |  |
| Reorganización del repositorio        | `xolobot-cognitive-core` en GitHub, documentación maestra, onboarding autónomo posible          | 16-18   |  |
| Resolución de conflicto `libfastcdr`  | Rebuild limpio documentado como procedimiento estándar post-`apt upgrade`                       | 16-18   |  |
| Acid Test en `ss-neuro`               | Gazebo Harmonic ejecutándose en instalación limpia Ubuntu 24.04; secuencia de agarre completada | 19-21   |  |
| Gestión de almacenamiento             | Liberación de 13 GB (97% → 81%); 529 GB auxiliares disponibles en `/mnt/datos`                  | 22-24   |  |
| Traspaso a Diego Vázquez              | Prerrequisitos diagnosticados, materiales entregados, onboarding iniciado                       | 22-24   |  |

El estado del proyecto al cierre del servicio social es el siguiente: la arquitectura cognitiva del Xolobot opera en ROS 2 Jazzy Jalisco con Gazebo Harmonic, su documentación permite la incorporación autónoma de nuevos integrantes, y el proceso de traspaso a la siguiente generación de alumnos ha sido formalmente iniciado con la orientación de la responsable del proyecto.